

U.D. Nº 6

ERROS E EXCEPCIÓNS. CÓDIGO ROBUSTO.

Nas unidades anteriores non reparamos moito nas mensaxes de erro, pero é habitual atopalos nos nosos programas. Hai basicamente dous tipos diferentes de erros: *erros de sintaxe e excepcións*.

1) Erros de sintaxe.

Tamén son coñecidos como erros de interpretación, e son comúns na fase de aprendizaxe da linguaxe. Por exemplo:

```
>>> while True print('Hello world')
      File "<stdin>", line 1
        while True print('Hello world')
                        ^^^^^
SyntaxError: invalid syntax
```

Neste caso o intérprete volve a escribirnos a liña cunhas frechas apuntando ao sitio onde detectou o erro. Non obstante, non sempre indica o sitio exacto que debe ser modificado. No exemplo anterior, o erro detectado en *print()* é debido aos *dous puntos(:)* omitidos xusto antes.

O nome do ficheiro (*stdin*) e o nº de liña son mostrados para que saibamos onde buscar.

2) Excepcións.

Aínda no caso de ter código sintacticamente correcto, un programa pode xerar un erro cando se intenta executar. Os erros detectados durante a execución chámanse excepcións, e non teñen porque dar ao traste coa execución do mesmo se os sabemos xestionar desde dentro do propio código.

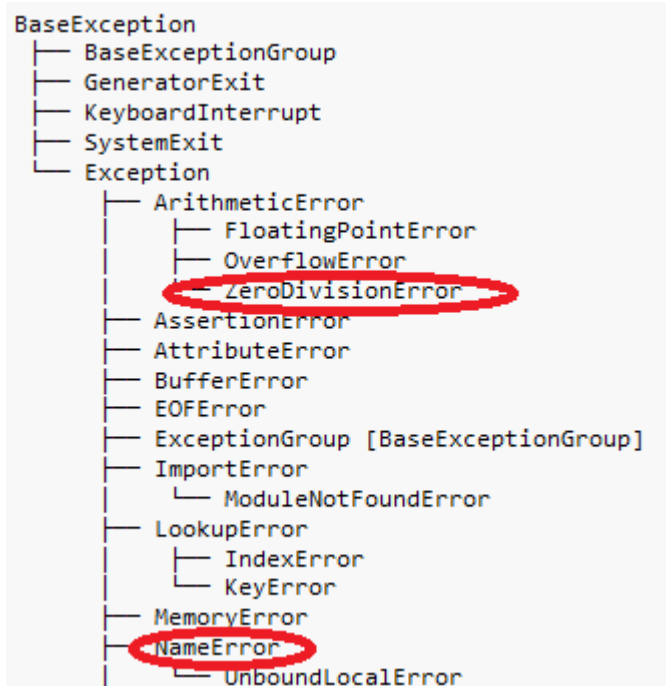
No caso de que suceda unha excepción e esta non sexa xestionada desde o código, veríamos erros como nos seguintes exemplos:

```

>>> 10 * (1/0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    10 * (1/0)
    ~^^
ZeroDivisionError: division by zero
>>> 4 + spam*3
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    4 + spam*3
    ~^^^
NameError: name 'spam' is not defined
>>> '2' + 2
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    '2' + 2
    ~~~~~~
TypeError: can only concatenate str (not "int") to str

```

Na última liña das mensaxes de erro indícase que foi o que pasou. Hai diferentes tipos de excepcións: *ZeroDivisionError*, *NameError*, *TypeError*, *ValueError*,...



- **ValueError:** cando o valor dun argumento é incorrecto.
- **TypeError:** cando o tipo de dato non é o esperado.
- **ZeroDivisionError:** cando intentas dividir por cero.
- **NameError:** cando se utiliza unha variable que non foi definida.

XESTIÓN DAS EXCEPCIÓNS

Para xestionar as excepcións dentro dos nosos programas podemos usar a sentencia:

try ... except

A sentencia *try ... except* funciona da seguinte maneira:

- O primeiro que fai é executar as liñas entre as palabras reservadas *try* e *except*, é dicir, executa a cláusula *try*.
- Se todo vai ben, a cláusula *except* omítese (no se executa), pero se ocorre unha excepción durante a execución do *try*, omítese o resto do *try* e compróbase se o tipo de excepción coincide co tipo indicado despois do *except*, en cuxo caso execútase a parte do código incluído no *except*, é dicir, execútase a cláusula *except* e despois continúa fora do *try-except*.

```
import math
print("*** CÁLCULO DA LONXITUDE DUNHA CIRCUNFERENCIA ***")
while True:
    try:
        radio = int(input("Introduce o radio:"))
        break
    except ValueError:
        print("Non é un número válido. Volve a intentalo.")

lonxitude = 2 * math.pi * radio
print(f"A lonxitude da circunferencia é igual a: {lonxitude}")

** CÁLCULO DA LONXITUDE DUNHA CIRCUNFERENCIA **
Introduce o radio: dd
Non é un número válido. Volve a intentalo.
Introduce o radio: 2
A lonxitude da circunferencia é igual a: 12.566370614359172
```

- Unha sentencia *try* pode ter máis dunha cláusula *except*, co obxectivo de manexar o erro de forma diferente segundo sexa o tipo de excepción. Como máximo só se executa unha das cláusulas *except*, e unicamente se manexan as excepcións sucedidas dentro do *try* (non as que ocorran dentro das outras *except*).

```
print("** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS **")
while True:
    try:
        persoas = int(input("Introduce o número de persoas:"))
        reparto = 100 / persoas
        break
    except ValueError:
        print("Non é un número válido. Volve a intentalo.")
    except ZeroDivisionError:
        print("Non é posible dividir entre cero persoas. Volve a intentalo.")

print(f"Cada persoa toca a {reparto} euros")
```

** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS **
Introduce o número de persoas: 0
Non é posible dividir entre cero persoas. Volve a intentalo.
Introduce o número de persoas: aa
Non é un número válido. Volve a intentalo.
Introduce o número de persoas: 4
Cada persoa toca a 25.0 euros

- Se ocorre un tipo de excepción que non coincide co indicado despois do primeiro *except*, buscará nas seguintes *except* do mesmo *try*, pero se non atopa ningún *except* que manexe a excepción, finalizará a execución do programa cunha mensaxe de erro.

```
print("*** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS ***")
while True:
    try:
        persoas = int(input("Introduce o número de persoas:"))
        reparto = 100 / persoas
        break
    except ValueError:
        print("Non é un número válido. Volve a intentalo.")
    except TypeError:
        print("Error de tipos.")

print(f"Cada persoa toca a {reparto} euros")

** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS **
Introduce o número de persoas: 0

-----
ZeroDivisionError                                Traceback (most recent call last)
Cell In[25], line 5
      3 try:
      4     persoas = int(input("Introduce o número de persoas:"))
----> 5     reparto = 100 / persoas
      6     break
      7 except ValueError:

ZeroDivisionError: division by zero
```

- Tamén se poden incluír varios tipos de excepcións dentro da mesma cláusula *except*, incluíndo o mesmo código para todos eses tipos.

```
print("*** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS ***")
while True:
    try:
        persoas = int(input("Introduce o número de persoas:"))
        reparto = 100 / persoas
        break
    except (ValueError, ZeroDivisionError):
        print("Non é un número válido. Volve a intentalo.")

print(f"Cada persoa toca a {reparto} euros")

** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS **
Introduce o número de persoas: aa
Non é un número válido. Volve a intentalo.
Introduce o número de persoas: 0
Non é un número válido. Volve a intentalo.
Introduce o número de persoas: 3
Cada persoa toca a 33.333333333333336 euros
```

- Os tipos de excepcións están definidos como clases, de tal maneira que todas herdan da clase base *BaseException*. Isto significa que si poñemos dito tipo na primeira cláusula *except*, atrapará todas as excepcións e xa nos se comprobarán as seguintes cláusulas *except*. A xerarquía de clases é a seguinte:

```

BaseException
├── BaseExceptionGroup
├── GeneratorExit
├── KeyboardInterrupt
├── SystemExit
├── Exception
│   ├── ArithmeticError
│   │   ├── FloatingPointError
│   │   ├── OverflowError
│   │   └── ZeroDivisionError
│   ├── AssertionError
│   ├── AttributeError
│   ├── BufferError
│   ├── EOFError
│   ├── ExceptionGroup [BaseExceptionGroup]
│   ├── ImportError
│   │   └── ModuleNotFoundError
│   ├── LookupError
│   │   ├── IndexError
│   │   └── KeyError
│   ├── MemoryError
│   ├── NameError
│   │   └── UnboundLocalError
│   ├── OSError
│   │   ├── BlockingIOError
│   │   ├── ChildProcessError
│   │   ├── ConnectionError
│   │   │   ├── BrokenPipeError
│   │   │   ├── ConnectionAbortedError
│   │   │   ├── ConnectionRefusedError
│   │   │   └── ConnectionResetError
│   │   ├── FileExistsError
│   │   ├── FileNotFoundError
│   │   ├── InterruptedError
│   │   ├── IsADirectoryError
│   │   ├── NotADirectoryError
│   │   ├── PermissionError
│   │   ├── ProcessLookupError
│   │   └── TimeoutError
│   ├── ReferenceError
│   ├── RuntimeError
│   │   ├── NotImplementedError
│   │   ├── PythonFinalizationError
│   │   └── RecursionError
│   ├── StopAsyncIteration
│   ├── StopIteration
│   ├── SyntaxError
│   │   ├── IndentationError
│   │   └── TabError
│   ├── SystemError
│   ├── TypeError
│   ├── ValueError
│   │   └── UnicodeError
│   │       ├── UnicodeDecodeError
│   │       ├── UnicodeEncodeError
│   │       └── UnicodeTranslateError
│   └── Warning

```

```

└─ Warning
    ├── BytesWarning
    ├── DeprecationWarning
    ├── EncodingWarning
    ├── FutureWarning
    ├── ImportWarning
    ├── PendingDeprecationWarning
    ├── ResourceWarning
    ├── RuntimeWarning
    ├── SyntaxWarning
    ├── UnicodeWarning
    └── UserWarning

```

```

import math
print("*** CÁLCULO DA LONXITUDE DUNHA CIRCUNFERENCIA ***")
while True:
    try:
        radio = int(input("Introduce o radio:"))
        break
    except BaseException:
        print("Erro xenérico")
    except ValueError:
        print("Non é un número válido. Volve a intentalo.")

lonxitude = 2 * math.pi * radio
print(f"A lonxitude da circunferencia é igual a: {lonxitude}")

** CÁLCULO DA LONXITUDE DUNHA CIRCUNFERENCIA **
Introduce o radio: aa
Erro xenérico
Introduce o radio: 2
A lonxitude da circunferencia é igual a: 12.566370614359172

```

try ... except ... else

- A sentencia *try-except* ten unha cláusula *else* opcional, a cal sitúase despois de todos os *except*. O código contido dentro do *else* executarase só no caso de que a cláusula *try* non tivera lanzada ningunha excepción.

```

print("*** PASO DE MILLAS A METROS ***")
try:
    n_millas = float(input("Introduce o número de millas:"))
except ValueError:
    print("Non é un número válido. Fin de programa")
else:
    n_metros = n_millas * 1609.34
    print(f"Equivalencia en metros {n_metros}")

** PASO DE MILLAS A METROS **
Introduce o número de millas: a
Non é un número válido. Fin de programa

```

- É mellor usar a cláusula *else* que poñer todo en *try*, pois evita a captura accidental dunha excepción non xerada polo código que queremos protexer. Ademais aporta máis claridade, pois separa a xestión de erros (*except*) da lóxica funcional (*else*).

<pre>print("*** PASO DE MILLAS A METROS **") try: n_millas = float(input("Introduce o número de millas:")) n_metros = n_millas* 1609.34 print(f"Equivalencia en metros {n_metros}") except ValueError: print("Non é un número válido. Fin de programa")</pre>	<pre>print("*** PASO DE MILLAS A METROS **") try: n_millas = float(input("Introduce o número de millas:")) except ValueError: print("Non é un número válido. Fin de programa") else: n_metros = n_millas* 1609.34 print(f"Equivalencia en metros {n_metros}")</pre>
<pre>** PASO DE MILLAS A METROS ** Introduce o número de millas: 2.5 Equivalencia en metros 4023.35</pre>	<pre>** PASO DE MILLAS A METROS ** Introduce o número de millas: 2.5 Equivalencia en metros 4023.35</pre>

- Os xestores de excepción (*except*) ademais de xestionar o que ocorre nas liñas de código da cláusula *try*, tamén xestionan excepcións que suceden dentro das funcións que son chamadas no código incluído en *try*.

```
def reparto(n):
    res = 100 / n
    return(res)

print("*** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS ***")
fin = False
while (not fin):
    try:
        persoas = int(input("Introduce o número de persoas:"))
        cantidad = reparto(persoas)
    except ValueError:
        print("Non é uan número válido. Volve a intentalo.")
    except ZeroDivisionError:
        print("Non se pode dividir entre cero persoas.")
    else:
        print(f"Cada persoa toca a {cantidad} euros")
        fin = True

** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS **
Introduce o número de persoas: a
Non é uan número válido. Volve a intentalo.
Introduce o número de persoas: 0
Non se pode dividir entre cero persoas.
Introduce o número de persoas: 2
Cada persoa toca a 50.0 euros
```


try ... except ... else ... finally

- A sentencia *try-except* ten outra cláusula opcional *finally*, a cal se executa ao final, antes de que toda a sentencia se complete (independentemente de que houbera ou non unha excepción).

```
try:
    persoas = int(input("Introduce o número de persoas:"))
    cantidad = 100 / persoas
except ValueError:
    print("Non é un número válido. Volve a intentalo.")
finally:
    print("FIN DO PROGRAMA")
```

```
Introduce o número de persoas: aa
Non é un número válido. Volve a intentalo.
FIN DO PROGRAMA
```

```
try:
    persoas = int(input("Introduce o número de persoas:"))
    cantidad = 100 / persoas
except ValueError:
    print("Non é un número válido. Volve a intentalo.")
finally:
    print("FIN DO PROGRAMA")
```

```
Introduce o número de persoas: 4
FIN DO PROGRAMA
```

Se houbo unha excepción e non foi xestionada desde unha cláusula *except*, relánzase despois da execución do código de *finally*. O mesmo sucede se a excepción se produce dentro dun *except* ou un *else*.

```
try:
    persoas = int(input("Introduce o número de persoas:"))
    cantidad = 100 / persoas
except ValueError:
    print("Non é un número válido. Volve a intentalo.")
finally:
    print("FIN DO PROGRAMA")
```

```
Introduce o número de persoas: 0
FIN DO PROGRAMA
```

```
-----
ZeroDivisionError                                Traceback (most recent call last)
Cell In[12], line 3
     1 try:
     2     persoas = int(input("Introduce o número de persoas:"))
----> 3     cantidad = 100 / persoas
     4 except ValueError:
     5     print("Non é un número válido. Volve a intentalo.")

ZeroDivisionError: division by zero
```

```
try:
    persoas = int(input("Introduce o número de persoas:"))
except ValueError:
    print("Non é un número válido. Volve a intentalo.")
else:
    cantidad = 100 / persoas
finally:
    print("FIN DO PROGRAMA")
```

```
Introduce o número de persoas: 0
FIN DO PROGRAMA
```

```
-----
ZeroDivisionError                                Traceback (most recent call last)
Cell In[16], line 6
     4     print("Non é un número válido. Volve a intentalo.")
     5 else:
----> 6     cantidad = 100 / persoas
     7 finally:
     8     print("FIN DO PROGRAMA")

ZeroDivisionError: division by zero
```

- Se o bloque *try* chega a unha sentencia **break**, **continue** ou **return**, a cláusula *finally* executarase antes da execución de dita sentencia.

```
import math
print("*** CÁLCULO DA LONXITUDE DUNHA CIRCUNFERENCIA **")
while True:
    try:
        radio = int(input("Introduce o radio:"))
    except ValueError:
        print("Non é un número válido.")
    else:
        lonxitude = 2 * math.pi * radio
        print(f"A lonxitude da circunferencia é igual a: {lonxitude}")
        break
    finally:
        print("Estou no finally")

** CÁLCULO DA LONXITUDE DUNHA CIRCUNFERENCIA **
Introduce o radio: aa
Non é un número válido.
Estou no finally
Introduce o radio: 2
A lonxitude da circunferencia é igual a: 12.566370614359172
Estou no finally
```

- O propósito da cláusula *finally* é definir accións de limpeza. En aplicacións reais, é útil para liberar recursos externos (como arquivos ou conexións de rede).

except ... as ...

- Mediante “as” podemos obter a mensaxe que Python asocia de forma automática a cada tipo de excepción. Bastaría con poñer “**as mensaxe_error**” na cláusula *except* despois do tipo de excepción.

```
print("*** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS **")
while True:
    try:
        persoas = int(input("Introduce o número de persoas:"))
        reparto = 100 / persoas
        break
    except ValueError as err1:
        print(err1)
    except ZeroDivisionError as err2:
        print(err2)

print(f"Cada persoa toca a {reparto} euros")

** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS **
Introduce o número de persoas: aa
invalid literal for int() with base 10: 'aa'
Introduce o número de persoas: 0
division by zero
Introduce o número de persoas: 2
Cada persoa toca a 50.0 euros
```

- Outra forma alternativa é recoller a mensaxe poñendo varios tipos de excepción na mesma cláusula *except*:

```
print("*** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS **")
while True:
    try:
        persoas = int(input("Introduce o número de persoas:"))
        reparto = 100 / persoas
    except (ValueError, ZeroDivisionError) as error:
        print(error)
    else:
        print(f"Cada persoa toca a {reparto} euros")
        break

** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS **
Introduce o número de persoas: aa
invalid literal for int() with base 10: 'aa'
Introduce o número de persoas: 0
division by zero
Introduce o número de persoas: 2
Cada persoa toca a 50.0 euros
```

PROGRAMACIÓN ROBUSTA. ERROS SEMÁNTICOS

Os erros semánticos son aqueles nos que o código é correcto (non hai erros sintácticos) e o programa se executa sen problemas (non hai excepcións), pero o resultado final do programa non é válido. Son os máis difíciles de detectar e/ou corrixir, pois a maioría das veces nin o programador nin os usuarios se decatan de que algo vai mal.

No último programa faise un reparto entre unha cantidade de persoas que o usuario introduce por teclado. Se dito usuario introduce un número negativo, o resultado do programa sería incorrecto, pois non ten sentido repartir unha cantidade entre -5 persoas, por exemplo. Trátase dun **erro semántico** derivado da utilización errónea do programa por parte do usuario.

A creación dun **programa robusto** fronte a todas as posibles entradas é unha faceta indispensable dentro a enxeñería do software.

- O seguinte código esixe mediante un condicional que o usuario introduza un número positivo, corrixindo o posible erro semántico. A cláusula *except* evita os formatos de entrada incorrectos ou cero.

```
print("*** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS ***")
while True:
    try:
        persoas = int(input("Introduce o número de persoas:"))
        reparto = 100 / persoas
    except ValueError:
        print("Non é un número válido. Volve a intentalo.")
    except ZeroDivisionError:
        print("Non se pode dividir entre cero persoas. Volve a intentalo")
    else:
        if (persoas < 0):
            print("O número de persoas non pode ser negativo. Volve a intentalo")
        else:
            print(f"Cada persoa toca a {reparto} euros")
            break

** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS **
Introduce o número de persoas: aa
Non é un número válido. Volve a intentalo.
Introduce o número de persoas: 0
Non se pode dividir entre cero persoas. Volve a intentalo
Introduce o número de persoas: -5
O número de persoas non pode ser negativo. Volve a intentalo
Introduce o número de persoas: 2
Cada persoa toca a 50.0 euros
```

raise TipoDeErro(mensaxe) --> (CREACIÓN MANUAL DE EXCEPCIÓNS)

- No exemplo anterior podemos considerar que o feito de que o usuario introduza un número negativo de persoas é algo pouco habitual (excepcional). En lugar de usar un condicional *if-else*, sería máis axeitado poder xerar manualmente unha excepción de entre os tipos de excepción nativos de Python. Para lograr o anterior dispoñemos da sentencia *raise*, onde a maiores podemos personalizar a mensaxe que se mostrará cando o *except* trate a excepción.
- Á hora de elixir o tipo de excepción, no exemplo consideraremos que *ValueError* é o que máis se axusta ao noso tipo de erro semántico.

```

print("*** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS ***")
while True:
    try:
        persoas = int(input("Introduce o número de persoas:"))
        reparto = 100 / persoas
        if (persoas < 0):
            raise ValueError("O nº de persoas non pode ser negativo")
        break
    except ValueError as error:
        print(error)
        print("Volve a intentalo.")
    except ZeroDivisionError:
        print("Non se pode dividir entre cero persoas. Volve a intentalo")
print(f"Cada persoa toca a {reparto} euros")

** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS **
Introduce o número de persoas: aa
invalid literal for int() with base 10: 'aa'
Volve a intentalo.
Introduce o número de persoas: 0
Non se pode dividir entre cero persoas. Volve a intentalo
Introduce o número de persoas: -5
O nº de persoas non pode ser negativo
Volve a intentalo.
Introduce o número de persoas: 4
Cada persoa toca a 25.0 euros

```

- Tamén se pode usar a sentencia *raise* para volver a lanzar a excepción despois de tela capturada nun *except*, co obxectivo de que sexa manexada nun contexto superior (para propagar o erro orixinal).

```

def reparto(n):
    try:
        res = 100 / n
    except ZeroDivisionError as err:
        print(err)
        raise
    else:
        return(res)

print("*** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS ***")
while True:
    try:
        persoas = int(input("Introduce o número de persoas:"))
        cantidad = reparto(persoas)
    except ValueError:
        print("Non é un número válido. Volve a intentalo.")
    except ZeroDivisionError:
        print("Non se pode dividir entre cero persoas")
    else:
        print(f"Cada persoa toca a {cantidad} euros")
        break

** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS **
Introduce o número de persoas: aa
Non é un número válido. Volve a intentalo.
Introduce o número de persoas: 0
division by zero
Non se pode dividir entre cero persoas
Introduce o número de persoas: 5
Cada persoa toca a 20.0 euros

```

- Se en lugar de lanzar unha excepción nativa de Python desexamos lanzar as nosas propias excepcións personalizadas, deberemos definir unha nova clase de excepción que herde da clase *Exception*. A continuación poderíamos lanzala con *raise* e despois tratala con *except*.

```
class NumeroPersoasNegativo(Exception):
    pass

print(""" REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS """)
while True:
    try:
        persoas = int(input("Introduce o número de persoas:"))
        reparto = 100 / persoas
        if (persoas < 0):
            raise NumeroPersoasNegativo("O nº de persoas non pode ser negativo")
        break
    except ValueError:
        print("Non é un número válido. Volve a intentalo.")
    except ZeroDivisionError:
        print("Non se pode dividir entre cero persoas")
    except NumeroPersoasNegativo as error:
        print(error)

print(f"Cada persoa toca a {reparto} euros")

** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS **
Introduce o número de persoas: aa
Non é un número válido. Volve a intentalo.
Introduce o número de persoas: 0
Non se pode dividir entre cero persoas
Introduce o número de persoas: -3
O nº de persoas non pode ser negativo
Introduce o número de persoas: 2
Cada persoa toca a 50.0 euros
```

- Definindo o método *__init__* para a nova clase podemos establecer unha mensaxe de erro por defecto, que se mostrará se non foi dado ao instanciar o tipo de erro.

```
class NumeroPersoasNegativo(Exception):
    def __init__(self, message="O nº de persoas non pode ser negativo"):
        self.message = message
        super().__init__(self.message)

print(""" REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS """)
while True:
    try:
        persoas = int(input("Introduce o número de persoas:"))
        reparto = 100 / persoas
        if (persoas < 0):
            raise NumeroPersoasNegativo()
        break
    except (ValueError, ZeroDivisionError, NumeroPersoasNegativo) as error:
        print(error)

print(f"Cada persoa toca a {reparto} euros")

** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS **
Introduce o número de persoas: aa
invalid literal for int() with base 10: 'aa'
Introduce o número de persoas: 0
division by zero
Introduce o número de persoas: -3
O nº de persoas non pode ser negativo
Introduce o número de persoas: 2
Cada persoa toca a 50.0 euros
```

- Finalmente, podemos construír a mensaxe de erro con máis argumentos como, por exemplo, o número de persoas. Para facer isto, hai que pasarlle o valor cando se instancia o novo tipo de erro.

```
class NumeroPersoasNegativo(Exception):
    def __init__(self, n_persoas, message="Non se permite un nº negativo"):
        self.n_persoas = n_persoas
        self.message = message
        super().__init__(self.message)

    def __str__(self):
        return f"Numero de persoas: {self.n_persoas} / {self.message}"

print("*** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS ***")
while True:
    try:
        persoas = int(input("Introduce o número de persoas:"))
        reparto = 100 / persoas
        if (persoas < 0):
            raise NumeroPersoasNegativo(persoas)
        break
    except (ValueError, ZeroDivisionError, NumeroPersoasNegativo) as error:
        print(error)

print(f"Cada persoa toca a {reparto} euros")

** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS **
Introduce o número de persoas: aa
invalid literal for int() with base 10: 'aa'
Introduce o número de persoas: 0
division by zero
Introduce o número de persoas: -3
Numero de persoas: -3 / Non se permite un nº negativo
Introduce o número de persoas: 2
Cada persoa toca a 50.0 euros
```

Fontes:

<https://docs.python.org/es/3/tutorial/errors.html>

https://www2.eii.uva.es/fund_inf/python/notebooks/07_Manejo_de_excepciones/Manejo_de_excepciones.html